

Module 12 – Version Control with Git

1. Introduction to Version Control

What is Version Control?

A **Version Control System (VCS)** is a software tool that records changes made to files over time. It allows developers to track modifications, restore previous versions, and collaborate efficiently without overwriting each other's work.

Instead of manually saving multiple copies of files (e.g., Project_Final_v1, Project_Final_v2), a VCS maintains a complete history of changes in an organized manner.

Purpose of Version Control

The primary purpose of version control is to manage changes to source code and other project files throughout the software development lifecycle.

It helps developers:

- Track every change made to a project.
- Restore earlier versions if needed.
- Work collaboratively without conflicts.
- Maintain a history of who made each change and when.
- Support parallel development through branching.

Benefits of Version Control

Using a Version Control System provides several advantages:

Tracks Change History

Every modification is recorded, making it easy to review or revert changes.

Facilitates Collaboration

Multiple developers can work on the same project simultaneously without interfering with each other's work.

Supports Backup and Recovery

Previous versions of files can be restored if code is accidentally deleted or modified incorrectly.

Enables Parallel Development

Developers can create separate branches to work on new features or bug fixes independently.

Improves Code Quality

Changes can be reviewed before being merged into the main project, reducing errors and maintaining coding standards.

Simplifies Project Management

Teams can monitor progress, identify contributors, and manage software releases more effectively.

Types of Version Control Systems

1. Local Version Control System (LVCS)

- Stores version history on a single local computer.
- Suitable for individual developers.
- Does not support team collaboration effectively.

Example: RCS (Revision Control System)

2. Centralized Version Control System (CVCS)

- Maintains a central server containing the project repository.
- Developers connect to the server to access and update files.
- If the server becomes unavailable, collaboration may be interrupted.

Examples: SVN (Subversion), CVS

3. Distributed Version Control System (DVCS)

- Every developer has a complete copy of the repository, including its full history.
- Changes can be committed locally without requiring a network connection.
- Synchronization with remote repositories occurs when desired.

Examples: Git, Mercurial

Git is the most widely used Distributed Version Control System today.

2. Understanding Git

What is Git?

Git is a free, open-source Distributed Version Control System (DVCS) designed to track changes in source code efficiently. It was created by **Linus Torvalds** in 2005 to support Linux kernel development.

Git enables developers to manage code changes, collaborate with teams, and maintain a complete project history.

Why Git is Popular

Git is widely adopted because it offers:

- High performance.
- Distributed architecture.
- Efficient branching and merging.
- Reliable change tracking.
- Strong support for collaborative development.
- Integration with platforms such as GitHub, GitLab, and Bitbucket.

Git as a Distributed Version Control System (DVCS)

Unlike centralized systems, Git stores the complete repository on every developer's machine.

This means developers can:

- Commit changes locally.
- View project history offline.
- Create branches without affecting others.
- Synchronize with remote repositories whenever needed.

This distributed model improves reliability, flexibility, and collaboration.

Git Components

Git manages files through three primary areas.

1. Working Directory

The **Working Directory** is the local folder where developers create, modify, or delete project files.

All changes are initially made here.

2. Staging Area (Index)

The **Staging Area** is an intermediate space where selected changes are prepared before being committed.

Developers choose which modifications should be included in the next commit.

3. Repository

The **Repository** stores the complete history of the project, including all commits, branches, and metadata.

Repositories can be:

- **Local Repository** – Stored on the developer's computer.
- **Remote Repository** – Hosted on platforms like GitHub or GitLab for collaboration.

Git Workflow

A typical Git workflow follows these steps:

1. Modify files in the **Working Directory**.
2. Stage changes using `git add`.
3. Save changes permanently using `git commit`.
4. Push commits to a remote repository using `git push`.
5. Pull updates from the remote repository using `git pull`.

3. Setting Up Git

Installing Git

To begin using Git:

1. Download Git from the official Git website.
2. Run the installer.
3. Accept the default installation settings (unless customization is required).
4. Verify the installation by opening a terminal and running:

```
git --version
```

If Git is installed correctly, the installed version number will be displayed.

Configuring Git

After installation, configure your identity so Git can associate commits with the correct author.

```
git config --global user.name "Your Name"
```

Sets the username for all repositories.

```
git config --global user.email "your@email.com"
```

Sets the email address associated with commits.

To verify the configuration:

```
git config --list
```

Creating a Git Repository

Initializing a New Repository

To convert an existing project folder into a Git repository:

```
git init
```

This command creates a hidden .git directory containing the repository metadata.

Cloning an Existing Repository

To copy an existing remote repository to your local machine:

```
git clone <repository-url>
```

The command downloads the project files along with the complete version history.

4. Basic Git Commands

git init

Initializes a new Git repository in the current directory.

```
git init
```

Use this command when starting version control for a new or existing project.

git clone

Creates a local copy of an existing remote repository.

```
git clone https://github.com/user/project.git
```

git add

Stages files for the next commit.

Stage a single file:

```
git add filename
```

Stage all modified files:

```
git add .
```

Wildcards can also be used, for example:

```
git add *.java
```

This stages all Java files in the current directory.

git commit

Saves the staged changes permanently in the repository history.

```
git commit -m "Added login functionality"
```

A good commit message should briefly describe the purpose of the changes.

git status

Displays the current state of the repository.

```
git status
```

It shows:

- Modified files
- Staged files
- Untracked files
- Current branch
- Repository status

This command is commonly used before committing changes.

git log

Displays the commit history.

```
git log
```

Useful options include:

```
git log --oneline
```

Displays each commit in a compact, single-line format.

```
git log --graph
```

Visualizes branch and merge history using an ASCII graph.

Reviewing the commit history helps developers understand project evolution and identify previous changes.

5. Branching and Merging

Introduction to Branching

A **branch** is an independent line of development within a Git repository.

Branches allow developers to work on new features, bug fixes, or experiments without affecting the main codebase.

The default branch is typically named **main** or **master**.

Creating a Branch

```
git branch feature-login
```

Creates a new branch named feature-login.

Switching Between Branches

```
git checkout feature-login
```

Or, using the modern command:

```
git switch feature-login
```

This changes the current working branch.

Creating and Switching in One Step

```
git checkout -b feature-login
```

or

```
git switch -c feature-login
```

These commands create the branch and switch to it immediately.

Merging Branches

After completing development on a branch, its changes can be integrated into another branch.

Example:

```
git checkout main  
git merge feature-login
```

Git attempts to combine the changes automatically.

Merge Conflicts

A **merge conflict** occurs when Git cannot automatically determine how to combine changes because the same part of a file has been modified differently in multiple branches.

To resolve a conflict:

1. Identify the conflicted files.
2. Edit the files to choose or combine the desired changes.
3. Save the files.
4. Stage the resolved files using `git add`.
5. Complete the merge with a new commit if required.

Resolving conflicts carefully helps preserve all necessary changes.

Branching Strategies

Feature Branch Workflow

Each new feature is developed in its own branch.

Advantages:

- Isolates development.
- Simplifies testing and code review.
- Prevents unstable code from affecting the main branch.

Release Branch Workflow

A dedicated branch is created to prepare a software release.

Developers can continue working on new features while release-specific fixes are applied independently.

Git Flow Workflow

Git Flow defines several long-lived branches:

- main – Production-ready code.
- develop – Ongoing development.
- feature – New features.
- release – Release preparation.
- hotfix – Urgent production fixes.

This workflow is commonly used in large or enterprise projects.

6. Remote Repositories

What is a Remote Repository?

A **remote repository** is a Git repository hosted on a server, enabling multiple developers to collaborate on the same project.

Popular hosting platforms include:

- GitHub
- GitLab
- Bitbucket

Adding a Remote Repository

To link a local repository to a remote repository:

```
git remote add origin <repository-url>
```

Here, origin is the conventional name for the primary remote.

A project may also have multiple remotes if it interacts with different repositories.

git push

Uploads local commits to the remote repository.

```
git push origin main
```

This command sends the commits from the local main branch to the corresponding branch on the remote.

git pull

Downloads the latest changes from the remote repository and merges them into the current local branch.

```
git pull origin main
```

Developers should pull frequently to stay synchronized with the team's latest work.

Remote Branches

Remote branches represent branches stored in the remote repository.

Developers can:

- Track remote branches.
- Create local branches based on remote branches.
- Push new local branches to the remote.

Keeping local and remote branches synchronized ensures smooth collaboration.

7. Collaborating with Git

Forking a Repository

A **fork** is a personal copy of another developer's repository created under your own account.

Forking is commonly used in:

- Open-source contributions.
- Projects where contributors do not have direct write access.

Changes made in the fork do not affect the original repository until they are proposed for integration.

Pull Requests

A **Pull Request (PR)** is a request to merge changes from one branch or fork into another branch.

A typical pull request process involves:

1. Creating a feature branch or fork.

2. Making and committing changes.
3. Pushing the branch to the remote repository.
4. Opening a pull request.
5. Reviewing and discussing the proposed changes.
6. Approving and merging the pull request.

Pull requests improve code quality by encouraging peer review before integration.

Git Collaboration Workflows

Centralized Workflow

- Developers work with a single shared repository.
- Changes are committed locally and pushed directly to the central repository.
- Suitable for small teams.

Feature Branch Workflow

- Every feature is developed in its own branch.
- Code is reviewed before being merged.
- Helps keep the main branch stable.

Forking Workflow

- Each contributor works in a personal fork of the repository.
- Pull requests are used to propose changes.
- Commonly adopted by open-source projects.

Git Flow Workflow

A structured workflow that separates development into multiple branch types:

- Main branch for production-ready code.
- Develop branch for ongoing work.
- Feature branches for new functionality.
- Release branches for preparing releases.
- Hotfix branches for urgent production fixes.

It provides clear organization for large teams and complex projects.

Best Practices for Using Git

- Commit changes frequently with clear, meaningful messages.
- Pull the latest changes before starting new work.
- Use feature branches instead of working directly on the main branch.

- Keep commits focused on a single logical change.
- Resolve merge conflicts promptly and carefully.
- Review code through pull requests before merging.
- Avoid committing sensitive information such as passwords or API keys.
- Keep the local repository synchronized with the remote repository.

Quick Revision Points

- A **Version Control System (VCS)** tracks changes to files and enables collaboration.
- Git is a **Distributed Version Control System (DVCS)** where every developer has a complete local copy of the repository.
- Git consists of three key areas: **Working Directory**, **Staging Area**, and **Repository**.
- Configure Git using `git config --global user.name` and `git config --global user.email`.
- `git init` creates a new repository, while `git clone` copies an existing repository.
- `git add` stages changes, `git commit` records them, `git status` shows the repository state, and `git log` displays commit history.
- Branches enable independent development, and `git merge` integrates changes back into another branch.
- Remote repositories are connected using `git remote add`; changes are uploaded with `git push` and downloaded with `git pull`.
- Forks and Pull Requests support collaborative development, especially in open-source projects.
- Common collaboration models include the **Centralized Workflow**, **Feature Branch Workflow**, **Forking Workflow**, and **Git Flow Workflow**.